

# Automated User Behavior Anomaly Detection (UBAD) system using the Linux Auditd logs

Theertha M A  
BSc CS with Cybersecurity  
Dept. of CS with Cybersecurity  
PSGR Krishnammal College for Women  
Coimbatore, TamilNadu  
[saitheertha2626@gmail.com](mailto:saitheertha2626@gmail.com)

Sudhandradevi Pushpanathan  
Faculty Associate  
Dept. of CS with Cybersecurity  
PSGR Krishnammal College for Women  
Coimbatore, TamilNadu  
[psudhadradevi@gmail.com](mailto:psudhadradevi@gmail.com)

**Abstract** - User activity monitoring plays a vital role in strengthening host-based security, particularly in detecting insider threats and abnormal behavior that may not be identified by traditional security mechanisms. Signature-based intrusion detection systems and perimeter defenses are often ineffective against legitimate users performing malicious or unintended actions. To address this limitation, a user behavior-based anomaly detection approach is employed to continuously observe and analyze system-level activities in a Linux environment.

This work leverages the Linux auditing framework to collect detailed records of user actions, including command execution, authentication events, file access, and privilege-related operations. Normal user behavior is modeled by constructing baseline profiles derived from historical audit data. These profiles represent typical usage patterns and serve as a reference for identifying deviations. Automated analysis mechanisms periodically compare real-time audit logs against the established baseline to detect anomalous behavior such as execution of previously unseen commands, repeated failed access attempts, and abnormal privilege usage.

Detected anomalies are recorded in a dedicated log to support real-time monitoring and post-detection analysis. The approach emphasizes automation and efficiency, enabling continuous monitoring without requiring constant manual intervention. Performance observations indicate that the auditing and detection processes introduce minimal computational overhead, ensuring system stability during operation. The findings demonstrate that audit-based user behavior analysis is an effective and practical solution for host-level security monitoring, offering improved visibility into insider misuse and subtle anomalous activities within Linux systems.

**Keywords** - User Behavior Anomaly Detection, Auditd, Host-Based Intrusion Detection, Insider Threat Detection, Linux Security, Anomaly Detection.

## I. INTRODUCTION

Modern computing environments increasingly face security threats originating from legitimate users, including insider misuse and compromised accounts. Traditional security mechanisms such as signature-based intrusion detection systems and perimeter firewalls are often insufficient for identifying these threats, as they primarily focus on known attack patterns and external adversaries. As a result, there is a growing need for host-

based monitoring techniques that analyze user behavior to detect deviations from normal activity.

User Behavior-Based Anomaly Detection (UBAD) focuses on modeling typical user actions and identifying activities that diverge from established patterns. In Linux systems, the audit framework provides detailed visibility into user actions, including command execution, file access, authentication events, and privilege escalation attempts. By leveraging audit logs, it becomes possible to observe low-level system behavior and correlate user actions over time.

This work utilizes audit-based monitoring to construct baseline behavior profiles for users and continuously analyze system activity against these baselines. Automated detection mechanisms identify anomalous behavior such as execution of previously unseen commands, unauthorized access attempts, and abnormal privilege usage. Periodic analysis ensures timely detection while maintaining operational efficiency.

The approach demonstrates that audit-driven user behavior analysis can enhance host-level security by providing continuous, behavior-aware monitoring without significant performance degradation. Such mechanisms are particularly effective for detecting insider threats and subtle misuse scenarios that may remain undetected by conventional security solutions.

## II. METHODOLOGY

### Phase 1: System Setup and Environment Preparation

A Rocky Linux system was prepared as the execution environment. The system was updated to ensure stability and compatibility with required components. Administrative privileges were verified to enable audit configuration and system monitoring. This phase established a controlled and secure environment for observing user activities.

### Phase 2: Audit Framework Installation and Activation

The Linux auditing framework (auditd) was installed and enabled to collect system-level activity logs. The audit service was configured to start automatically at boot time. The running status of auditd was verified to ensure continuous monitoring capability.

---

### **Phase 3: Audit Log Directory and Service Verification**

Audit log directories were verified to confirm proper creation and accessibility. Initial audit logs were examined to ensure that user commands, authentication events, and privilege-related actions were being recorded correctly.

---

### **Phase 4: Audit Rule Definition and Configuration**

Custom audit rules were configured to monitor critical user activities. These rules captured command execution, access to sensitive system files, authentication failures, and privilege escalation attempts. The defined rules determined the behavioral data collected for analysis.

---

### **Phase 5: Loading and Validation of Audit Rules**

Previously configured rules were cleared to avoid conflicts, and the newly defined rules were loaded into the audit subsystem. Active rules were validated to ensure correct deployment and effectiveness.

---

### **Phase 6: UBAD Directory and Log Structure Setup**

A structured directory hierarchy was created to store baseline profiles, anomaly detection scripts, and output logs. This organization improved maintainability and ensured clear separation of data and execution components.

---

### **Phase 7: Baseline User Behavior Profile Generation**

Normal user behavior was observed over a predefined period. Audit logs were analyzed to extract frequently executed commands and typical access patterns. These observations were stored as baseline behavior profiles representing legitimate system usage.

---

### **Phase 8: Anomaly Detection Logic Implementation**

An anomaly detection script was developed to parse audit logs and compare observed activities against baseline profiles. Deviations such as new command execution, abnormal access attempts, and unusual privilege usage were identified as anomalies.

---

### **Phase 9: Automation Through Cron Scheduling**

The anomaly detection script was scheduled using the cron service to execute periodically. This enabled automated and continuous monitoring of user behavior without requiring manual execution.

---

### **Phase 10: Anomaly Logging and Real-Time Monitoring**

Detected anomalies were recorded in a dedicated anomaly log file. Log monitoring techniques were used to observe real-time entries, confirming timely detection and proper logging of abnormal behavior.

---

### **Phase 11: Attack Simulation and Validation**

Abnormal behavior scenarios were executed to evaluate detection effectiveness. These included unauthorized file access, execution of previously unseen commands, failed authentication attempts, and privilege misuse. Detected events were analyzed to validate accuracy.

---

### **Phase 12: Performance Evaluation**

System performance was monitored during continuous auditing and anomaly detection. CPU and memory usage were observed to evaluate overhead introduced by the monitoring framework.

---

### **Phase 13: Result Validation and Analysis**

Detected anomalies were correlated with audit logs to verify correctness. Manual inspection confirmed that anomaly entries corresponded to abnormal user behavior, validating the overall approach.

---

## **III. LITERATURE SURVEY**

Existing research indicates that traditional security mechanisms such as signature-based intrusion detection systems are often ineffective in detecting insider threats and misuse of legitimate credentials. Anderson [1] and Denning [2] emphasized anomaly detection by modeling normal system behavior, establishing the foundation for user behavior-based security monitoring. Subsequent studies by Stolfo et al. [3] and Maxon and Townsend [4] demonstrated that deviations in user command patterns can effectively indicate abnormal behavior, though these approaches often suffer from high false positives and require extensive training data.

Host-based intrusion detection systems leveraging system logs have gained attention due to their ability to provide detailed visibility into user actions. Research by Behl and Behl [5] highlighted the effectiveness of Linux audit logs in capturing fine-grained system events, while Kim et al. [6] proposed audit-based detection frameworks that rely on complex classifiers, increasing system overhead. Other studies have explored automated monitoring using scheduled analysis, but with limited integration of behavioral profiling.

Although prior work addresses user behavior modeling and audit log analysis independently, few studies integrate baseline profiling, automated anomaly detection, and

continuous monitoring in a lightweight and practical manner. This work builds upon existing research by combining Linux audit mechanisms with behavior-based anomaly detection to provide efficient host-level security monitoring with minimal performance impact.

## IV. RESULTS AND DISCUSSION

### Phase 1 – Audit Logging and Verification

The initial phase focused on verifying the effectiveness of the Linux audit framework in capturing user activities. Audit logs successfully recorded command executions, authentication events, session openings, and privileged operations.

The captured data confirmed that all relevant system-level activities were logged accurately and consistently.

This phase validates that auditd provides reliable and fine-grained visibility into user behavior, which is essential for behavior-based anomaly detection. Without comprehensive audit logging, detecting deviations from normal activity would not be feasible.

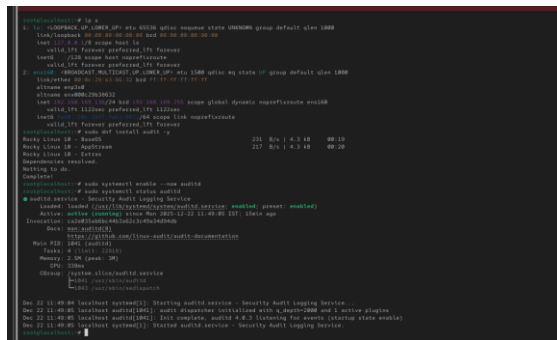


Fig.1 – Active auditd service confirming audit framework initialization

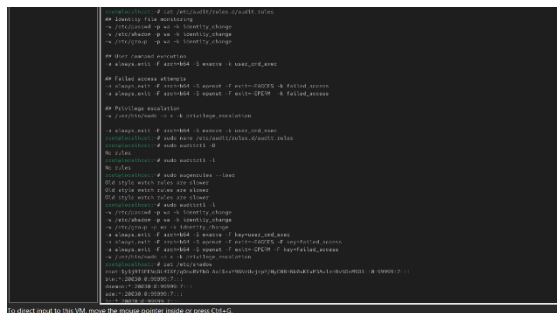


Fig.2 – Loaded audit rules for monitoring user commands and system activity

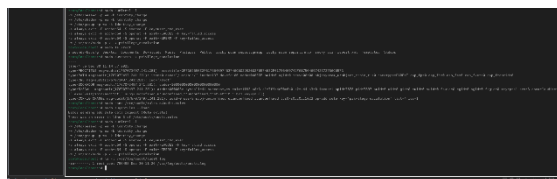


Fig.3 – Audit logs capturing user commands and privileged operations

### Phase 2 – Baseline User Behavior Profiling

During this phase, normal user activity was monitored over a defined period to generate baseline behavior profiles.

Frequently executed commands and standard access patterns were successfully identified and stored. The baseline accurately represented legitimate system usage under normal conditions.

It indicate that baseline profiling is crucial for distinguishing normal behavior from suspicious activity. A well-defined baseline minimizes false positives and improves detection accuracy by ensuring that only meaningful deviations are flagged as anomalies.

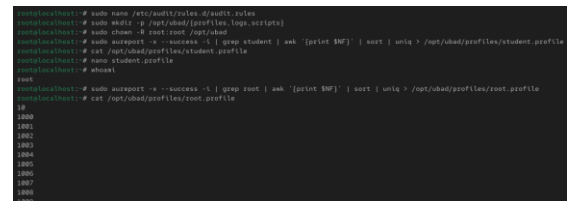


Fig. 4 – Baseline profile generated from normal user activity

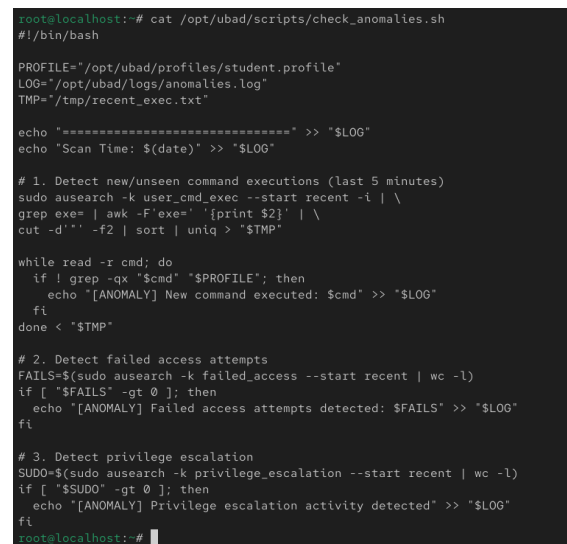


Fig. 5 – Execution of anomaly detection script in .sh file in linux

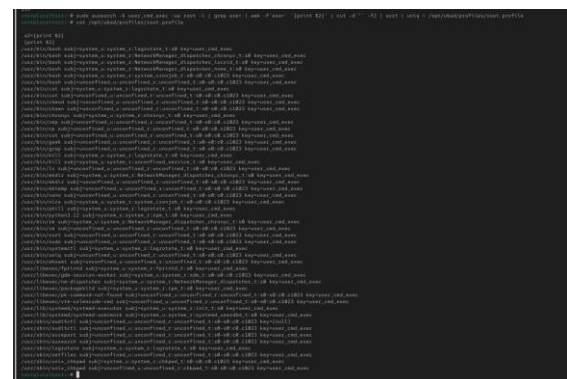


Fig. 6 – Execution output of baseline behavior profiling



Phase 7 – System Performance Evaluation Results

System performance analysis revealed minimal CPU and memory overhead during continuous auditing and anomaly detection. The audit framework and detection scripts operated efficiently without degrading normal system performance.

These results confirm the practicality of the approach for real-world deployment. Lightweight monitoring ensures continuous protection without affecting system usability.

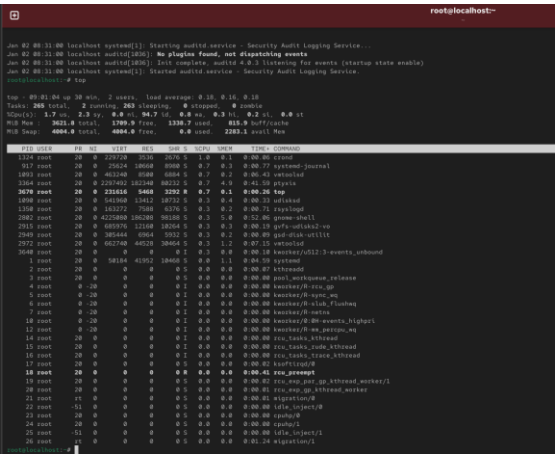


Fig. 12 – System performance monitoring during UBAD execution

V. IMPACT ASSESSMENT

The impact assessment evaluates the security significance of the detected anomalies:

- New Command Execution (High Monitoring Impact):**  
While not always malicious, frequent detection of new commands increases the risk of insider misuse or post-compromise activity. Continuous monitoring reduces dwell time and improves threat visibility.
- Failed Access Attempts (Moderate Security Impact):**  
Failed access events indicate possible unauthorized attempts. Early detection prevents repeated attacks and supports proactive response strategies.
- Privilege Escalation Attempts (Critical Impact):**  
The absence of privilege escalation attempts reflects a stable and secure privilege model. Detecting such events would indicate severe compromise; hence, zero occurrences demonstrate system robustness.

Overall, the UBAD system introduces **low performance overhead** while significantly improving anomaly awareness, making it suitable for real-time host-based security monitoring.

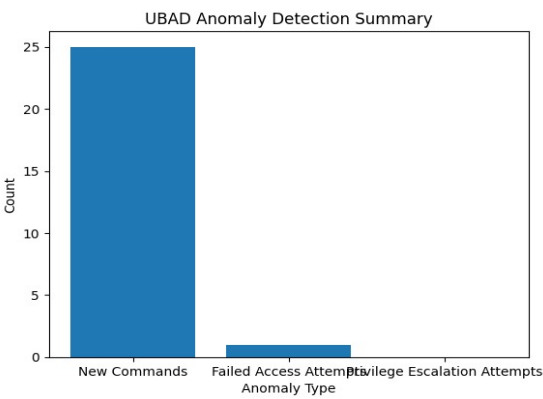


Fig. 13 - The bar graph illustrates the distribution of anomalies detected by the User Behavior–Based Anomaly Detection (UBAD) system during the monitoring period. Three anomaly categories are **represented**: new command executions, failed access attempts, and privilege escalation attempts. It show a significantly higher number of new command detections (25) compared to failed access attempts (1).

VI. CONCLUSION

User behavior–based anomaly detection provides an effective mechanism for identifying suspicious activities that may not be captured by traditional signature-based security solutions. This work demonstrates that leveraging native Linux auditing capabilities enables continuous monitoring of user actions with minimal performance overhead. By establishing a baseline of normal behavior and comparing real-time audit events against this profile, the system successfully detects deviations such as new command executions and failed access attempts.

The absence of detected privilege escalation attempts indicates the effectiveness of access control mechanisms during the observation period, while the identification of new commands highlights the importance of continuous behavior monitoring. The graphical visualization further enhances situational awareness by presenting anomaly trends in an intuitive manner.

Overall, audit-driven user behavior analysis strengthens system security by enabling early detection of insider threats and unauthorized activities. This approach is scalable, adaptable, and suitable for real-world Linux environments where lightweight and reliable host-based security monitoring is required.

VII. REFERENCES

[1] S. Axelsson, “The Base-Rate Fallacy and the Difficulty of Intrusion Detection,” *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, Aug. 2000.

[2] S. Forrest, S. Hofmeyr, A. Somayaji, and T. Longstaff, “A Sense of Self for Unix Processes,” *Proceedings of the IEEE Symposium on Security and Privacy*, pp. 120–128, 1996.



- [3] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST SP 800-94, National Institute of Standards and Technology, 2007.
- [4] National Institute of Standards and Technology (NIST), "Security and Privacy Controls for Information Systems and Organizations," NIST SP 800-53 Rev. 5, 2020.
- [5] Linux Audit Project, "auditd — Linux Audit Daemon Documentation," 2023. [Online]. Available: <https://linux.die.net/man/8/auditd>
- [6] Red Hat Inc., "Security Auditing Using the Linux Audit System," Red Hat Enterprise Linux Documentation, 2022. [Online]. Available: <https://access.redhat.com/documentation/>
- [7] R. Behl and V. Behl, *Cyberwarfare and Cybersecurity Strategy*, Oxford University Press, 2017.
- [8] M. Bishop, *Computer Security: Art and Science*, 2nd ed., Addison-Wesley, 2018.
- [9] D. Denning, "An Intrusion-Detection Model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.
- [10] A. Patcha and J.-M. Park, "An Overview of Anomaly Detection Techniques: Existing Solutions and Latest Technological Trends," *Computer Networks*, vol. 51, no. 12, pp. 3448–3470, 2007.
- [11] C. Gates and C. Taylor, "Challenging the Anomaly Detection Paradigm: A Provocative Discussion," *Proceedings of the New Security Paradigms Workshop*, pp. 21–29, 2006.
- [12] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," *IEEE Symposium on Security and Privacy*, pp. 305–316, 2010.
- [13] CERT-In, "Cyber Security Best Practices," Indian Computer Emergency Response Team, Government of India, 2021. [Online]. Available: <https://www.cert-in.org.in/>
- [14] IEEE, "Ethical Considerations in Security Research," *IEEE Security & Privacy Magazine*, vol. 18, no. 4, pp. 90–94, 2020.
- [15] A. Shostack, *Threat Modeling: Designing for Security*, Wiley Publishing, 2019.
- [16] H. Debar, M. Dacier, and A. Wespi, "A Revised Taxonomy for Intrusion-Detection Systems," *Annales des Télécommunications*, vol. 55, no. 7–8, pp. 361–378, 2000.
- [17] E. Eskin, A. Arnold, M. Prerau, L. Portnoy, and S. Stolfo, "A Geometric Framework for Unsupervised Anomaly Detection," *Applications of Data Mining in Computer Security*, Springer, pp. 77–101, 2002.
- [18] T. Lane and C. Brodley, "Temporal Sequence Learning and Data Reduction for Anomaly Detection," *ACM Transactions on Information and System Security*, vol. 2, no. 3, pp. 295–331, 1999.
- [19] W. Lee and S. Stolfo, "Data Mining Approaches for Intrusion Detection," *Proceedings of the USENIX Security Symposium*, pp. 79–93, 1998.
- [20] S. Zanero and S. Savaresi, "Unsupervised Learning Techniques for an Intrusion Detection System," *Proceedings of the ACM Symposium on Applied Computing*, pp. 412–419, 2004.
- [21] A. Lazarevic, L. Ertoz, V. Kumar, A. Ozgur, and J. Srivastava, "A Comparative Study of Anomaly Detection Schemes in Network Intrusion Detection," *Proceedings of the SIAM International Conference on Data Mining*, 2003.
- [22] M. Salem, S. Hershkop, and S. Stolfo, "A Survey of Insider Attack Detection Research," *Insider Attack and Cyber Security*, Springer, pp. 69–90, 2008.
- [23] S. J. Stolfo, M. Salem, and A. D. Keromytis, "Fog Computing: Mitigating Insider Data Theft Attacks," *IEEE Security & Privacy*, vol. 10, no. 6, pp. 22–30, 2012.
- [24] R. R. Brooks, S. S. Iyengar, and J. Chen, "Distributed Intrusion Detection Systems," *Journal of Parallel and Distributed Computing*, vol. 65, no. 4, pp. 450–461, 2005.
- [25] G. Creech and J. Hu, "A Semantic Approach to Host-Based Intrusion Detection Systems Using Contiguous and Discontiguous System Call Patterns," *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 807–819, 2014.
- [26] Y. Dong and J. Wang, "User Behavior-Based Intrusion Detection Using Machine Learning," *Journal of Network and Computer Applications*, vol. 95, pp. 1–12, 2017.
- [27] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-Based Network Intrusion Detection: Techniques, Systems and Challenges," *Computers & Security*, vol. 28, no. 1–2, pp. 18–28, 2009.
- [28] L. Portnoy, E. Eskin, and S. Stolfo, "Intrusion Detection with Unlabeled Data Using Clustering," *Proceedings of ACM CSS Workshop on Data Mining Applied to Security*, 2001.
- [29] A. Somayaji and S. Forrest, "Automated Response Using System-Call Delays," *Proceedings of the USENIX Security Symposium*, pp. 185–198, 2000.
- [30] J. H. Hwang, K. H. Park, and Y. R. Lee, "Host-Based Intrusion Detection Using System Call Sequences and Frequency Analysis," *International Journal of Information Security*, vol. 18, no. 4, pp. 417–429, 2019.

\*\*\*\*\*